Aran Learn 26: SaaS and Multi-Tenancy

For: everyone. Aasif and Arun first (they build the Aran Cloud side). **Time:** about 1 hour with practice. **Before this:** Learn 8 (web APIs and proxies) and Learn 25 part 2 (tenant isolation).

Today Aran runs as one program that one team starts on one machine. The next step is to sell it as a **SaaS** product: banks sign up, get keys, send their traffic to Aran Cloud and watch the results on a dashboard. This doc explains the words and ideas behind that change, using the parts of Aran Cloud that already exist (`aran_cloud/main.py`).

---

1. What SaaS means

SaaS (software as a service) means the customer does not install the software. They use it over the internet and pay for how much they use. Gmail, Razorpay and Stripe are SaaS products.

Installed software	**SaaS**
Each bank runs its own copy	One service runs for every bank
The bank upgrades it	We upgrade it for everyone at once
Paid once, per licence	Paid per month, by plan or by use
The bank sees only its own copy	We see all customers, so isolation is our job

The last row is the hard part. In SaaS, many customers share the same servers, so one customer must never see another's data.

---

2. Tenants, workspaces and users

A **tenant** is one paying customer of the service. For Aran, a tenant is a bank. Aran Cloud calls the tenant's space a **workspace**.

```
Aran Cloud
├── Workspace: Bank A (tenant_id = t_a1…)
│   ├── API keys      (one per system: core banking, UPI switch, chat bot)
│   ├── Users         (security officer, auditor, developer)
│   └── Data          (tokens, usage events, receipts, audit log)
└── Workspace: Bank B (tenant_id = t_b7…)
    └── …the same, never mixed with Bank A
```

Every row Aran Cloud stores carries a `tenant_id` , and every query filters on it:

```
# aran_cloud/main.py: the dashboard only ever reads the caller's rows
events = conn.execute(
```

```
    "SELECT * FROM usage_events WHERE tenant_id = ? ORDER BY id DESC", (tenant_id,)
).fetchall()
```

If a query forgets the `WHERE tenant_id = ?`, Bank A could read Bank B's numbers. That is why the cloud has a test file just for this (`aran_cloud/test_cloud_security.py`).

Three ways to keep tenants apart

Model	How	Good	Bad
Shared tables	One database, a `tenant_id` column	Cheap, simple	One missed filter leaks data
Schema per tenant	One database, a schema per bank	Clearer walls	More to migrate
Database per tenant	A whole database per bank	Strongest wall	Costly, slow to sign up

Aran Cloud uses **shared tables** today. Big banks often ask for a dedicated database or region, so we plan for it as a paid plan.

---

3. API keys and scopes

A system talks to a SaaS service with an **API key**: a long random secret sent in a header.

```
curl -H "Authorization: Bearer aran_live_…" https://cloud.aran.example/v1/protect -d
'{"text": "…"}'
```

Rules Aran Cloud already follows:

- The key is shown **once**. The server keeps only its **SHA-256 hash**, so a stolen database does not give out working keys.
- Each key has **scopes**: what it is allowed to do (`protect`, `detokenise`, `erase`, `dashboard` …). The UPI switch gets `protect`; only the fraud team's tool gets `detokenise`. This is the **least privilege** idea.
- Keys can be **revoked** at once (`DELETE /v1/keys/{id}`).
- Each key has a **rate limit** (300 calls a minute), so one runaway program cannot slow down every other bank.

People (not programs) sign in with a username and password or the bank's own login (**SSO**). Each person has a **role**: admin, security officer, auditor, developer. A role is a ready-made set of scopes.

---

4. Metering, plans and billing

Metering means counting what each tenant uses. Aran Cloud writes one `usage_events` row per call, with the tenant, the time, how many identifiers were protected and how long it took.

A **plan** sets the price and the limits:

Plan	Price idea	Limits
Sandbox	Free	10,000 calls a month, test data only
Growth	Per million transactions	Shared cloud, 99.9% uptime
Enterprise	Yearly contract	Own database, India region only, 99.95% uptime, SSO

When a tenant passes its limit, the service can refuse (**hard limit**, HTTP 429) or keep going and bill more (**soft limit**). Banks prefer soft limits: a payment must never fail because of a quota.

---

5. Onboarding: the first ten minutes

A good SaaS gets a new customer to a working call fast:

1. Sign up, which creates a workspace and an admin key (`POST /v1/workspaces`).
2. Copy the key, install the SDK (`aran_sdk/aran.py`), send one test call.
3. See that call appear on the dashboard within seconds.
4. Invite teammates, create keys per system, pick a plan.

---

6. What a bank will ask before it signs

Banks are strict customers. Expect these questions:

- **Where is my data?** RBI rules say payment data must stay in India (**data localisation**). We pick an India region.
- **Who at Aran can see it?** Ideally no one: we keep tokens and keys, and the raw value only when the bank asks for a vault.
- **What if you go down?** An **SLA** (service level agreement) promises uptime, and Aran must fail in a way that does not stop payments (see Learn 27).
- **Can I prove what happened?** The audit log and signed erasure receipts answer this.

Exercises

1. Start Aran Cloud and create two workspaces with `POST /v1/workspaces` . Call `/v1/dashboard` with the first key. Can you find any way to see the second workspace's numbers?
2. Create a key with only the `protect` scope. Call `/v1/detokenise` with it. Which HTTP status comes back, and why is that the right one?
3. Open `cloud.db` with `sqlite3` and look at the `api_keys` table. Why can you not use what you see there to call the API?
4. Our free plan has 10,000 calls a month. A bank does 2 crore UPI payments a month. Which plan fits, and should the limit be hard or soft?

Words to know

Word	Meaning
SaaS	Software used over the internet and paid for by use, not installed.
Tenant	One paying customer of a shared service (here, a bank).
Workspace	The space in Aran Cloud that holds one tenant's keys, users and data.
Multi-tenant	One running service that serves many tenants at once.
API key	A long secret a program sends to prove which tenant it belongs to.
Scope	One thing a key is allowed to do.
Least privilege	Give each key and person only the scopes they need.
Role	A named set of scopes for a person (admin, auditor…).
SSO	Single sign-on: people log in with their bank's own account.
Metering	Counting each tenant's use, so it can be shown and billed.
Plan	A price with its limits.
Rate limit	The most calls a key may make in a time window.
SLA	A written promise of uptime and support.
Data localisation	Keeping data inside one country, as RBI requires for payment data.